Politecnico di Milano
Numerical Methods for Partial Differential Equations — A.Y. 2025/2026
Prof. Alfio Quarteroni — Dr. Michele Bucelli

# Adaptive Finite Elements for a Linear Parabolic Problem

**Alessandro Russi, Francesco Maria Reggio, and Seyed Vahid Ghayoomie**

## 1. Introduction

Partial Differential Equations (PDEs) are fundamental mathematical models for phenomena across various scientific and engineering disciplines. While analytical solutions are often intractable for complex problems, numerical methods, particularly the *Finite Element Method (FEM)*, offer robust approximation strategies. For time-dependent problems, such as the heat equation, the computational domain spans both space and time, presenting challenges in achieving accuracy and efficiency with uniform discretizations.

Adaptive methods represent a significant advancement in numerical simulation, allowing for dynamic adjustment of spatial and temporal resolutions based on estimates of the solution error. This targeted refinement concentrates computational resources where they are most needed, for example, near sharp fronts, singularities, or regions of high solution gradients, leading to more accurate results with optimized computational cost compared to global uniform meshes [1].

This theoretical study focuses on the linear parabolic problem, specifically the heat equation, with an emphasis on its strong and weak formulations, mathematical analysis, numerical discretization via FEM coupled with time-stepping schemes, and the theoretical underpinnings of error estimation crucial for adaptive algorithms. The goal is to lay the groundwork for developing an adaptive formulation that dynamically refines the space-time grid based on *a posteriori* error indicators, a strategy extensively discussed in the context of *a posteriori* error estimates for parabolic problems [1].

## 2. Mathematical Formulation

We consider a linear parabolic problem, often referred to as the heat equation, defined on a spatial domain $\Omega \subset \mathbb{R}^d$ (typically $d = 2$ or $d = 3$) over a time interval $(0, T)$. The problem describes the evolution of an unknown function $u(x, t)$ that depends on both spatial coordinates $x$ and time $t$.

### Strong Formulation

The strong form of the governing equation is given by:

$$\frac{\partial u}{\partial t} - \nabla \cdot (\mu \nabla u) = f \quad \text{in } \Omega \times (0,T) \quad (P.1)$$

The spatial domain $\Omega$ is assumed to be a convex polygon of $\mathbb{R}^2$ [1]. The problem is complemented by the following auxiliary conditions:

- **Boundary Conditions (BCs):** Homogeneous Neumann conditions are applied over the entire spatial boundary $\partial\Omega$ for all $t \in (0,T)$:

  $$\mu \nabla u \cdot n = 0 \quad \text{on } \partial\Omega \times (0,T) \quad (P.2)$$

  Here, $n$ is the outward unit normal vector to $\partial\Omega$. This condition implies no heat flux across the boundary.

- **Initial Condition (IC):** The initial state of the system is prescribed as zero throughout the domain $\Omega$ at $t = 0$:

  $$u(x,0) = 0 \quad \text{in } \Omega \times \{0\} \quad (P.3)$$

The parameters and forcing term are defined as follows:

- $\mu$: Diffusion coefficient, which is constant and set to $\mu = 1$.
- $f(x,t)$: Forcing term, representing a sequence of impulses localized in space and time. It is defined as a separable function of space and time:

  $$f(x,t) = g(t)h(x)$$

  where

  $$g(t) = \frac{\exp(-a\cos(2N\pi t))}{\exp(a)}$$

  and

  $$h(x) = \exp\left(-\frac{(x-x_0)^2}{\sigma^2}\right)$$

  with $a > 0$, $N \in \mathbb{N}$, $x_0 \in \Omega$, and $\sigma > 0$.

The overall computational domain is the space-time cylinder $Q_T = \Omega \times (0,T]$.

### Weak Formulation

To enable numerical solution, especially for solutions that may not possess the high regularity required for the strong form, we derive the weak (*variational*) formulation of the problem. This involves multiplying the PDE by a spatially-dependent test function and integrating over the spatial domain.

Let $V$ be a suitable functional space for the solution $u$ and test functions $v$. We seek $u(t,\cdot) \in V$ such that for all $v \in V$, the integral form of the PDE is satisfied. The space $V$ will typically be a Sobolev space, as discussed below.

1. **Multiply by a Test Function and Integrate:** Multiply equation (P.1) by a test function $v(x) \in V$ and integrate over $\Omega$:

   $$\int_\Omega \left(\frac{\partial u}{\partial t} - \nabla \cdot (\mu \nabla u)\right) v \, dx = \int_\Omega f v \, dx \quad \forall v \in V, \text{ a.e. } t \in (0,T)$$

This can be split into two terms:

$$\int_\Omega \frac{\partial u}{\partial t} v \, dx - \int_\Omega \nabla \cdot (\mu \nabla u) v \, dx = \int_\Omega f v \, dx$$

2. **Apply Green's First Identity (Integration by Parts):** The second term on the left-hand side involves a divergence. We apply Green's First Identity (also known as the Divergence Theorem) to this term:

$$-\int_\Omega \nabla \cdot (\mu \nabla u) v \, dx = \int_\Omega \mu \nabla u \cdot \nabla v \, dx - \int_{\partial\Omega} (\mu \nabla u \cdot n) v \, ds$$

3. **Incorporate Boundary Conditions:** The boundary integral term is simplified by the given homogeneous Neumann boundary condition (P.2):

$$\int_{\partial\Omega} (\mu \nabla u \cdot n) v \, ds = \int_{\partial\Omega} (0) v \, ds = 0$$

Thus, the boundary integral vanishes. This implies that the Neumann boundary conditions are "natural" boundary conditions in the variational formulation, as they are implicitly satisfied without further restrictions on the test function space.

4. **The Weak Formulation:** Combining these steps, the time-continuous weak formulation of the problem is: Find $u : (0, T) \to V$ such that $u(x, 0) = 0$ and for all $v \in V$:

$$\int_\Omega \frac{\partial u}{\partial t} v \, dx + \int_\Omega \mu \nabla u \cdot \nabla v \, dx = \int_\Omega f v \, dx \quad \forall v \in V, \text{ a.e. } t \in (0, T) \quad (W.1)$$

5. **Functional Spaces:** For the terms in (W.1) to be well-defined, $u$ and $v$ must have square-integrable first spatial derivatives. The standard choice for such problems is the Sobolev space $H^1(\Omega)$. Since there are no Dirichlet boundary conditions constraining the test function values to zero on the boundary, the test space $V$ is simply $H^1(\Omega)$.

More precisely, the solution $u$ is sought in the space $W = \{w \in L^2(0, T; H^1(\Omega)) \text{ and } \partial w/\partial t \in L^2(0, T; H^{-1}(\Omega))\}$ [1]. The initial condition $u(\cdot, 0) = u^0$ is understood in the $L^2(\Omega)$ sense if $u \in \mathcal{C}^0([0, T]; L^2(\Omega))$, which is a property of functions in $W$ [1].

We can express (W.1) using the mass matrix-like and stiffness matrix-like terms:

$$\left( \frac{\partial u}{\partial t}, v \right)_{L^2(\Omega)} + A(u, v) = (f, v)_{L^2(\Omega)} \quad \forall v \in V, \text{ a.e. } t \in (0, T)$$

where $A(u, v) = \int_\Omega \mu \nabla u \cdot \nabla v \, dx$ is the bilinear form and $(f, v)_{L^2(\Omega)} = \int_\Omega f v \, dx$ is the linear functional.

## Mathematical Analysis

The mathematical analysis for parabolic problems involves establishing the existence, uniqueness, and stability of the weak solution. These properties are critical to guarantee that the problem is *well-posed* and that numerical approximations are meaningful. The analysis often draws on functional analysis tools, such as the Lax-Milgram Lemma for the spatial elliptic operator, extended to the time-dependent setting via energy methods.

1. **Continuity of the Bilinear Form $A(u,v)$:** The bilinear form $A(u,v) = \int_\Omega \mu \nabla u \cdot \nabla v \, dx$ must be continuous (bounded) on $V \times V$. For $V = H^1(\Omega)$, using the Cauchy-Schwarz inequality and assuming $\mu$ is bounded (here $\mu = 1$):

$$|A(u,v)| = \left| \int_\Omega \mu \nabla u \cdot \nabla v \, dx \right| \leq \mu_{\max} \int_\Omega |\nabla u||\nabla v| \, dx \leq \mu_{\max} \|\nabla u\|_{L^2(\Omega)} \|\nabla v\|_{L^2(\Omega)}$$

Since $\|\nabla u\|_{L^2(\Omega)} \leq \|u\|_{H^1(\Omega)}$ (where $\|u\|^2_{H^1(\Omega)} = \|u\|^2_{L^2(\Omega)} + \|\nabla u\|^2_{L^2(\Omega)}$), we have:

$$|A(u,v)| \leq \mu_{\max} \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}$$

This proves continuity with $M = \mu_{\max}$ (here $M = 1$).

2. **Coercivity of the Bilinear Form $A(v,v)$:** The bilinear form $A(v,v)$ must be coercive. For the pure diffusion term $A(v,v) = \int_\Omega \mu |\nabla v|^2 \, dx$, with homogeneous Neumann boundary conditions on the entire boundary, $A(v,v)$ only involves the gradient term. This means $A(v,v) = 0$ for any constant function $v \neq 0$. Therefore, $A(v,v)$ is not coercive with respect to the full $H^1(\Omega)$ norm directly.

   However, for parabolic problems, a weaker coercivity condition is often sufficient, usually called **weak coercivity**. This states that there exists a constant $\lambda \geq 0$ such that $A(V,V) + \lambda \|V\|^2_{L^2(\Omega)} \geq \alpha \|V\|^2_V$ for some $\alpha > 0$. This modified form effectively makes the bilinear form coercive on $H^1(\Omega)$. The non-trivial $L^2$ term (the time derivative) in the parabolic equation helps establish well-posedness even when the spatial operator is only weakly coercive.

3. **Boundedness of the Linear Functional $F(v)$:** The linear functional $F(v) = \int_\Omega f v \, dx$ must be bounded (continuous) on $V$. Using the Cauchy-Schwarz inequality:

$$|F(v)| = \left| \int_\Omega f v \, dx \right| \leq \|f\|_{L^2(\Omega)} \|v\|_{L^2(\Omega)}$$

Since $\|v\|_{L^2(\Omega)} \leq \|v\|_{H^1(\Omega)}$, we have:

$$|F(v)| \leq \|f\|_{L^2(\Omega)} \|v\|_{H^1(\Omega)}$$

This proves boundedness with $C_F = \|f\|_{L^2(\Omega)}$.

4. **Existence, Uniqueness, and Stability:** For the parabolic heat equation with an $L^2$ initial condition and $L^2$ forcing, it is a standard result that a unique weak solution exists in the space $W = \{w \in L^2(0,T; H^1(\Omega)) \text{ and } \partial w/\partial t \in L^2(0,T; H^{-1}(\Omega))\}$ [1, 2]. Moreover, the solution $u(\cdot,t)$ belongs to $\mathcal{C}^0([0,T]; L^2(\Omega))$ [1].

   The stability of the solution is demonstrated through energy estimates. For the semi-discrete problem, the solution norm is bounded by the data. If $F = 0$, the $L^2$ norm of the solution at time $t$ is bounded by its initial condition, and the time-integrated $H^1$ semi-norm is also bounded:

$$\|U_h(t)\|^2_{L^2(\Omega)} + \alpha \int_0^t \|\nabla U_h(\tau)\|^2_{L^2(\Omega)} d\tau \leq \|U_0^h\|^2_{L^2(\Omega)}$$

This implies that the solution is stable and does not grow unboundedly.

## 3. Numerical Discretization

The weak formulation (W.1) still represents a continuous problem in both space and time. To obtain a computable solution, both spatial and temporal dimensions must be discretized.

### Spatial Semi-Discretization: Finite Element Method (FEM)

We discretize the spatial domain $\Omega$ using FEM.

1. **Finite Element Space:** We introduce a finite-dimensional subspace $V_h \subset V$ spanned by a basis of continuous, piecewise polynomial functions $\{\phi_j(x)\}_{j=1}^{N_h}$. Following the theoretical framework, one typically employs continuous, piecewise linear ($P_1$) or higher-order polynomial ($P_r$) finite elements [1].

2. **Approximate Solution:** The solution $u(x,t)$ is approximated by $u_h(x,t)$ in $V_h$:

$$u_h(x,t) = \sum_{j=1}^{N_h} U_j(t)\phi_j(x)$$

   where $U_j(t)$ are time-dependent coefficients.

3. **Galerkin Procedure:** Substituting $u_h$ into (W.1) and choosing test functions $v = \phi_i(x)$ for each $i = 1, \ldots, N_h$, we obtain a system of Ordinary Differential Equations (ODEs) in time, known as the semi-discrete system:

$$\sum_{j=1}^{N_h} \frac{dU_j}{dt} \int_\Omega \phi_j \phi_i \, dx + \sum_{j=1}^{N_h} U_j(t) \int_\Omega \mu \nabla\phi_j \cdot \nabla\phi_i \, dx = \int_\Omega f\phi_i \, dx$$

   This system is expressed in matrix form:

$$M\frac{d\mathbf{U}}{dt} + A\mathbf{U}(t) = \mathbf{F}(t) \quad (SD.1)$$

   where $\mathbf{U}(t) = [U_1(t), \ldots, U_{N_h}(t)]^T$ is the vector of nodal unknowns, and:

   - **Mass Matrix** $M$: $M_{ij} = \int_\Omega \phi_j\phi_i \, dx$.
   - **Stiffness Matrix** $A$: $A_{ij} = \int_\Omega \mu\nabla\phi_j \cdot \nabla\phi_i \, dx$.
   - **Load Vector** $\mathbf{F}(t)$: $F_i(t) = \int_\Omega f(x,t)\phi_i \, dx$.

The initial condition for the ODE system is $\mathbf{U}(0) = \mathbf{0}$, obtained by interpolating $u(x,0) = 0$.

### Temporal Discretization: Theta Method

To solve the semi-discrete system (SD.1), we discretize the time variable. We introduce a partition of $(0,T)$ into subintervals $(t^n - t^{n-1})$ and define time levels $0 = t_0 < t_1 < \cdots < t_N = T$. Let $\Delta t_n = t_n - t_{n-1}$ be the time step. We seek an approximation $\mathbf{U}^n \approx \mathbf{U}(t^n)$.

The **Theta-method** is a one-step finite difference scheme that approximates the time derivative and weights the spatial terms between $t^n$ and $t^{n+1}$. For an exhaustive and rigorous treatment of the stability and convergence properties of this family of methods applied to parabolic equations, we refer to Quarteroni [2]. The scheme for advancing from $\mathbf{U}^n$ to $\mathbf{U}^{n+1}$ is:

$$M\left(\frac{\mathbf{U}^{n+1} - \mathbf{U}^n}{\Delta t}\right) + A\left[\theta\mathbf{U}^{n+1} + (1-\theta)\mathbf{U}^n\right] = \theta\mathbf{F}^{n+1} + (1-\theta)\mathbf{F}^n \quad (FD.1)$$

where $\mathbf{F}^{n+1} = \mathbf{F}(t^{n+1})$ and $\mathbf{F}^n = \mathbf{F}(t^n)$.

Rearranging to solve for $\mathbf{U}^{n+1}$ at each time step, we get a linear algebraic system:

$$\mathbf{K}\mathbf{U}^{n+1} = \mathbf{R}^n$$

where

$$\mathbf{K} = \frac{1}{\Delta t}M + \theta A \quad (FD.2)$$

and

$$\mathbf{R}^n = \left(\frac{1}{\Delta t}M - (1-\theta)A\right)\mathbf{U}^n + \theta\mathbf{F}^{n+1} + (1-\theta)\mathbf{F}^n \quad (FD.3)$$

Following [1], we utilize the **Euler implicit (Backward Euler) scheme** for time discretization. This corresponds to choosing $\theta = 1$.

For $\theta = 1$:

- The scheme is first-order accurate in time ($\mathcal{O}(\Delta t)$).
- It is unconditionally stable, meaning it remains bounded for any choice of $\Delta t > 0$.

The fully discrete scheme for Backward Euler is:

$$\left(\frac{1}{\Delta t}M + A\right)\mathbf{U}^{n+1} = \frac{1}{\Delta t}M\mathbf{U}^n + \mathbf{F}^{n+1} \quad (FD.4)$$

At each time step, a linear system with matrix $K = \frac{1}{\Delta t}M + A$ is solved for $\mathbf{U}^{n+1}$. Since $M$ and $A$ are constant (as $\mu$ is constant), $K$ only needs to be assembled and factorized once if $\Delta t$ is fixed, significantly reducing computational cost.

### Theoretical Error Estimates

The accuracy of the fully discrete solution $u_{h\tau}(x,t)$ (where $h$ denotes spatial discretization and $\tau$ denotes temporal discretization) is determined by both the spatial and temporal errors.

For a linear parabolic problem solved with FEM of polynomial degree $r$ in space and a $\theta$-method of order $Q(\theta)$ in time, the error is typically bounded by:

$$\|u(t_n) - u_{h\tau}(t_n)\|_{norm} \leq C\left(H^r + (\Delta t)^{Q(\theta)}\right) \quad \text{(Error.1)}$$

where $H$ is the mesh size, and $C$ is a constant dependent on the exact solution's regularity but independent of $H$ and $\Delta t$.

Given the proposed scheme (FEM of degree $r$, and Backward Euler, i.e., $Q(\theta) = 1$ for $\theta \neq 1/2$), the theoretical error estimate for the exact solution $u$ and the numerical approximation $u_{h\tau}$ would be:

$$\|u - u_{h\tau}\|_{norm} \leq C(H + \Delta t)$$

This indicates first-order convergence in both space and time.

Detailed *a posteriori* error estimates for the error $e = u - u_{h\tau}$ in the $L^2$ in time, $H^1$ in space norm are established in [1]. The error estimator is comprised of three main contributions: $\eta$, $\epsilon$, and $\gamma$ [1].

1. **Spatial Residual Error ($\eta^n$):** This term measures the residual of the PDE equation in each element $K$ and jump of the normal derivative across edges $l$:

$$(\eta_K^n)^2 = \int_{t^{n-1}}^{t^n} \{|K| \|\Pi_K^n (f - \frac{\partial u_{h\tau}}{\partial t})\|_{0,K}^2 + \frac{1}{2} \sum_{l \in E_K^n} |l| \|\Pi_l^n (J_l^n)\|_{0,l}^2\} dt \quad \text{(Error.2)}$$

This is similar to error estimators for elliptic problems, capturing the spatial discretization error, particularly due to the piecewise linear approximation and local non-satisfaction of the PDE [1].

2. **Temporal Error from Solution Difference ($\epsilon^n$):** This term estimates the error introduced by the time discretization scheme based on the difference between solutions at consecutive time steps:

$$(\epsilon_K^n)^2 = \int_{t^{n-1}}^{t^n} |\nabla (u_h^n - u_h^{n-1})\|_{0,K}^2 dt \quad \text{(Error.3)}$$

This component measures the effect of large solution changes between time levels [1].

3. **Temporal Error from Forcing Term Approximation ($\gamma^n$):** This term quantifies the error due to approximating the time-dependent forcing function $f$ with its value at $t^n$ (or some average):

$$(\gamma_K^n)^2 = \int_{t^{n-1}}^{t^n} \|f - f^n\|_{0,K}^2 dt \quad \text{(Error.4)}$$

Here $f^n = f(\cdot, t^n)$ [1]. This captures the error from not accurately resolving the time-dependence of the source term.

The total error estimators $\eta, \epsilon, \gamma$ are summed over all elements and time steps [1].

**Upper and Lower Bounds:** Theoretical bounds on the error $e = u - u_{h\tau}$ are provided in [1].

- **Upper Bound (Theorem 2.1):** Under certain assumptions (regular meshes, nested triangulations, $f \in H^1((0,T) \times \Omega)$), for $h$ small enough, the error is bounded above:

$$\|e(\cdot, t^n)\|_{0,\Omega}^2 + \int_{t^{n-1}}^{t^n} |\nabla e\|_{0,\Omega}^2 dt \leq \|e(\cdot, t^{n-1})\|_{0,\Omega}^2 + C((\eta^n)^2 + (\epsilon^n)^2 + (\gamma^n)^2 + h^4 |f|_{1,(t^{n-1}, t^n) \times \Omega}^2) \quad \text{(Error.5)}$$

Summing over time steps, the global error follows $\int_0^T \|\nabla e\|_{0,\Omega}^2 dt \leq C(\eta^2 + \epsilon^2 + \gamma^2)$ [1].

- **Lower Bound (Theorem 2.2):** Conversely, the error estimator itself is bounded by the true error, confirming its reliability:

$$(\eta^n)^2 \leq C(\int_{t^{n-1}}^{t^n} \|\nabla e\|_{0,\Omega}^2 dt + (\epsilon^n)^2 + h^4 |f|_{1,(t^{n-1}, t^n) \times \Omega}^2) \quad \text{(Error.6)}$$

And globally, $\eta^2 \leq C(\int_0^T \|\nabla e\|_{0,\Omega}^2 dt + \epsilon^2)$ [1].

These bounds demonstrate that the introduced error estimator terms effectively capture the overall discretization error in both space and time. This framework is essential for adaptive algorithms, as it provides a computable measure of the error for refining the mesh and adjusting the time step [1].

# 4. Implementation Details

The numerical resolution of the formulated parabolic problem is accomplished through a specialized C++ implementation utilizing the deal.II finite element library. The computational strategy is inherently *Object-Oriented* (OOP), encapsulating the problem definition, discretization parameters, and functional logic within a primary `Heat` class. This encapsulation facilitates data hiding and modularity. The forcing term $f(x,t)$, given its separable nature $g(t)h(x)$, is structured using customized classes (`g_function` and `h_function`) that inherit from the deal.II `Function` base class, allowing for transparent evaluation at arbitrary coordinates and temporal instances.

To achieve high computational performance, the solver integrates distributed memory parallelism via the *Message Passing Interface* (MPI). The spatial domain is partitioned across multiple processors using `parallel::distributed::Triangulation`, the p4est-backed triangulation class. This choice is not incidental: deal.II's other MPI-parallel triangulation, `parallel::fullydistributed::Triangulation`, only supports a *static* partition built once (typically from an externally generated mesh file) and does not support local mesh refinement after construction. Since this project's central goal is *spatial* adaptivity, `parallel::distributed::Triangulation` is the only one of the two that can actually satisfy the requirement. Its natural companion is `FE_Q<dim>`, the Lagrange finite element on hexahedra/quadrilaterals — deal.II's simplex elements (`FE_SimplexP`, needed if the mesh were tetrahedral) explicitly do not support local $h$-refinement either. Consequently, the mesh here is generated programmatically as a hexahedral grid via `GridGenerator::subdivided_hyper_cube(mesh, n_elements, 0.0, 1.0)` on the unit cube $\Omega = (0,1)^3$, rather than imported from an external `.msh` file. Furthermore, the implementation relies on the Trilinos framework for distributed linear algebra structures, employing `TrilinosWrappers::SparseMatrix` and `TrilinosWrappers::MPI::Vector` to manage the global system across the cluster.

### Assembly and Preconditioning

At every accepted time step, the global system matrix and right-hand side are rebuilt from scratch by a single `assemble()` routine that loops over locally owned cells and directly evaluates, via Gauss quadrature, the full $\theta$-weighted bilinear form of (FD.4) — the mass term $\frac{1}{\Delta t}(\phi_i, \phi_j)$, the diffusion term $\theta\,\mu(\nabla\phi_i, \nabla\phi_j)$, and the forcing/history contributions on the right-hand side, evaluating $f$ at both $t_n$ and $t_{n+1}$ inside the same cell loop:

```
const double mu_loc     = mu(fe_values.quadrature_point(q));
const double f_old_loc  = f(fe_values.quadrature_point(q), time);
const double f_new_loc  = f(fe_values.quadrature_point(q), time + delta_t);

cell_matrix(i, j) += (1.0 / delta_t) * fe_values.shape_value(i, q)
                     * fe_values.shape_value(j, q) * fe_values.JxW(q);
cell_matrix(i, j) += theta * mu_loc
                     * scalar_product(fe_values.shape_grad(i, q),
                                      fe_values.shape_grad(j, q)) * fe_values.JxW(q);

cell_rhs(i) += (theta * f_new_loc + (1.0 - theta) * f_old_loc)
               * fe_values.shape_value(i, q) * fe_values.JxW(q);
```

Because the mesh itself changes shape every few steps (§4.4), the sparsity pattern and the matrix it is built on are not stable enough across the whole run to make a persistent, once-only

pre-assembly of $M$ and $A$ straightforward; re-assembling the full bilinear form every step is the simpler, more robust choice adopted here, at the cost of some redundant quadrature work between refinement cycles. The resulting linear system is solved with the Conjugate Gradient method (`SolverCG`) preconditioned with a symmetric successive over-relaxation preconditioner (`TrilinosWrappers::PreconditionSSOR`), which keeps the iteration count low (typically under 10) for this diffusion-dominated, well-conditioned operator without the added setup cost of an algebraic multigrid hierarchy.

### Non-Adaptive Baseline

A rigorous evaluation of any adaptive scheme calls for a non-adaptive baseline run under otherwise identical conditions — the same mesh generator, the same assembly and solver code, the same $\theta$ and forcing term, but with a fixed mesh and a fixed $\Delta t$ for the whole simulation. `Heat` now exposes a single boolean constructor parameter, `adaptive` (default `true`, so every pre-existing call site is unaffected), that guards both mechanisms at once: `run()` branches between the existing accept/reject/grow $\Delta t$ loop with periodic `refine_mesh()` calls (§4.4) when `true`, and a single always-accepted, fixed-$\Delta t$ step with no refinement when `false`. A helper, `run_basename()`, derives every output filename (`output-`, `mass_balance-`, `convergence-`) from the mesh size and this flag together, so adaptive and non-adaptive runs — and different mesh sizes — never collide on disk. The executable's own CLI exposes this as `mode = adaptive|nonadaptive` in a deal.II `ParameterHandler`-driven `.prm` file (alongside `run = default|convergence`, mesh/time-stepping parameters, and the three tolerances of §4.3), replacing the previous argument-less binary with one whose entire configuration is inspectable and version-controllable as plain text, without recompiling.

Comparing the two variants' peak solution *value* at $T = 1$ is deceptive: adaptive gives 0.0010571, non-adaptive gives 0.0010490 — only 0.8% apart, wrongly suggesting AMR barely matters. The real story is in mesh resolution near the source and in the field's *gradient*, not its raw value (a classical FEM fact: coarse elements underestimate gradients/curvature far more than they underestimate the value itself), as Figure 1 shows.

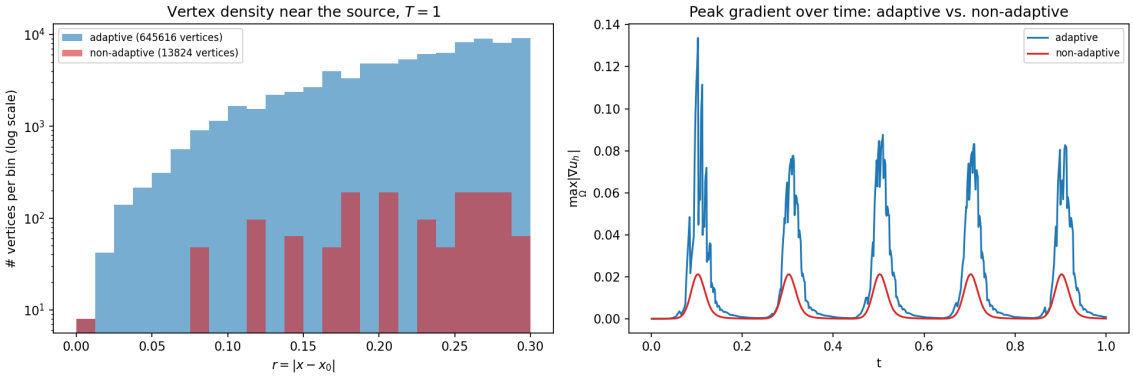



Figure 1: Left: radial vertex-density histogram at $T = 1$ (log scale) — adaptive has 645,616 vertices smoothly spread across $r = |x - x_0|$; non-adaptive has a fixed 13,824 (the unrefined base mesh, unchanged for the whole run) concentrated in sparse, evenly-spaced bins. Right: $\max_\Omega |\nabla u_h|(t)$ for both variants across the whole run — adaptive tracks a much sharper, taller peak at every pulse ($\approx 0.133$ at the first, $\approx 6\times$ non-adaptive's flat $\approx 0.021$ ceiling; $\approx 0.078$–$0.088$ at later pulses, $\approx 3.5$–$4\times$ non-adaptive's still-flat ceiling).

The uniform mesh structurally cannot resolve the steep gradient near the source, regardless of how close the raw solution values happen to land — precisely the comparison a non-adaptive baseline is meant to make possible.

### Temporal Adaptivity (k-adaptivity)

The main experiment uses the Backward Euler scheme ($\theta = 1.0$); the same code path is also exercised with $\theta = 0.5$ (Crank-Nicolson) in the manufactured-solution validation of §5.3. At every step, `compute_time_error_indicator()` evaluates a genuine domain integral combining two of Picasso's three error contributions (§3.3): a term $\epsilon^2 = \int_\Omega \Delta t \, \|\nabla u_h^{n+1} - \nabla u_h^n\|^2$ tracking how much the gradient changed over the step (matching Error.3's structure), and a term $\gamma^2$ quantifying how much the forcing $f$ varies *within* the substep — evaluated with a 3-point Gauss-Legendre rule in time rather than just comparing $f$ at the two endpoints, so a pulse that turns on and off between $t_n$ and $t_{n+1}$ is not missed (matching Error.4's structure):

```
const Tensor<1, dim> diff_grad = grad_new[q] - grad_old[q];
eps_sq += delta_t * diff_grad.norm_square() * fe_values.JxW(q);

const double f_new_here = f(x_q, time + delta_t);
double gamma_q = 0.0;
for (unsigned int g = 0; g < gauss_pts.size(); ++g) {
  const double t_g  = time + gauss_pts[g] * delta_t;
  const double diff = f(x_q, t_g) - f_new_here;
  gamma_q += gauss_wts[g] * diff * diff;
}
gamma_sq += gamma_q * delta_t * fe_values.JxW(q);
// ... summed over all cells and quadrature points, then reduced across MPI ranks
return std::sqrt(eps_sq + gamma_sq);
```

This combined indicator is compared against thresholds `tol_time_max`$= 1 \times 10^{-4}$ and `tol_time_min`$= 2 \times 10^{-6}$ (both, along with `dt_max`, promoted from hardcoded class constants to constructor parameters, so different callers can use different tolerances — the manufactured-solution check of §5.3, whose smooth forcing has no sharp feature to miss, uses looser values, $5{\times}10^{-3}$/$1{\times}10^{-4}$/`0.1`, than the pulsed problem's tight ones). If the indicator exceeds `tol_time_max`, the step is rejected, $\Delta t$ is halved (down to a floor `dt_min`$= 10^{-5}$), and the system is re-solved from the last accepted state. If it falls below `tol_time_min`, $\Delta t$ is grown by a factor 1.2 (up to a ceiling `dt_max`$= 0.01$) once the step is accepted, letting the solver take large strides through the quiescent periods between pulses. The ceiling is deliberately kept comparable to the forcing pulse's own width: `compute_time_error_indicator()`'s $\gamma$ term only samples the forcing at 3 fixed Gauss points inside each candidate step, so a $\Delta t$ allowed to grow much wider than a pulse risks stepping over it without any sample point landing near its peak — in which case $\gamma$ reads as (falsely) small and the step is accepted regardless of how tight `tol_time_max` is. Tightening the tolerance alone cannot fix that detection blind spot; only bounding `dt_max` to the feature width gives a hard guarantee on the worst-case miss distance.

The two mechanisms interact in a way that is worth stating explicitly, since it was found only after a first fix addressed `dt_max` alone: tightening `dt_max` without also tightening `tol_time_max`/`tol_time_min` makes the tolerance-based grow/reject logic go dormant, since a comparatively loose tolerance combined with a small `dt_max` lets $\Delta t$ climb straight to the (now low) ceiling and stay there almost permanently, with `dt_max` becoming the only effective control

rather than a backstop for a genuinely reactive one. Figure 3 (current values) shows the fix confirmed: a real sawtooth, growing during each quiet inter-pulse window and dropping sharply near every pulse, rather than a flat line pinned at the ceiling.

### Spatial Adaptivity (h-adaptivity)

Spatial resolution is re-evaluated every 5 accepted temporal integration steps. As in the temporal case, the solver uses the `KellyErrorEstimator` as an efficient proxy for the spatial residual $\eta_K$, based on jumps of the normal derivative across inter-element faces.

Based on the distribution of these error indicators, the mesh is modified with a *fixed-fraction* strategy: the 60% of cells with the largest indicator are flagged for refinement, and the 40% with the smallest are flagged for coarsening. Refinement is additionally capped at a maximum of `max_grid_level`= 2 levels beyond the base mesh, to keep the 3D cell count (which grows 8-fold per level, per cell) tractable.

Because `parallel::distributed::Triangulation` redistributes cells across MPI ranks on refinement, the solution is preserved with the p4est-aware `parallel::distributed::SolutionTransfer`, whose interface differs from the serial `SolutionTransfer`: it takes a vector of *pointers* to the fields being transferred (here both `solution` and `solution_old`, so both time levels survive a refinement event), and its `interpolate()` call only needs the output vectors, not the originals:

```
Vector<float> estimated_error_per_cell(mesh.n_active_cells());
KellyErrorEstimator<dim>::estimate(dof_handler, QGauss<dim - 1>(r + 1), {},
                                   solution, estimated_error_per_cell);

parallel::distributed::GridRefinement::refine_and_coarsen_fixed_fraction(
  mesh, estimated_error_per_cell, 0.6, 0.4);

if (mesh.n_levels() > max_grid_level)
  for (const auto &cell : mesh.active_cell_iterators_on_level(max_grid_level))
    cell->clear_refine_flag();

mesh.prepare_coarsening_and_refinement();

parallel::distributed::SolutionTransfer<dim, TrilinosWrappers::MPI::Vector>
  sol_trans(dof_handler);
std::vector<const TrilinosWrappers::MPI::Vector *> transfer_in = {&solution, &solution_old};
sol_trans.prepare_for_coarsening_and_refinement(transfer_in);

mesh.execute_coarsening_and_refinement();
setup_system(); // re-distributes DoFs on the new mesh

TrilinosWrappers::MPI::Vector interpolated_new(solution_owned), interpolated_old(solution_owned_old)
std::vector<TrilinosWrappers::MPI::Vector *> transfer_out = {&interpolated_new, &interpolated_old};
sol_trans.interpolate(transfer_out);
```

# 5. Numerical Experiments

## Experimental Setup

All experiments use the pulsed forcing term $f(x,t) = g(t)h(x)$ with $a = 5$, $N = 5$ (five pulses over $t \in (0,1]$), $\sigma = 0.1$, $x_0 = (0.5, 0.5, 0.5)$, constant diffusivity $\mu = 1$, final time $T = 1$, and the Backward Euler scheme ($\theta = 1$). The domain is the unit cube $\Omega = (0,1)^3$, discretized with degree-1 (`FE_Q<3>(1)`) hexahedral elements, and `max_grid_level`= 2 caps the refinement depth. All runs use `TrilinosWrappers::PreconditionSSOR` with the MPI/Trilinos stack described in §4, executed inside the project's Docker toolchain.

## Convergence Study

Since no closed-form solution exists for the pulsed problem, spatial/temporal convergence is assessed against a fine-mesh reference: an adaptive run with base resolution $N_{el} = 12$ is treated as ground truth, and its solution is sampled (via `Functions::FEFieldFunction`) at the same physical points as three coarser adaptive sweep runs, $N_{el} \in \{3, 6, 9\}$, to compute the $L^2$ and $H^1$ errors. This particular sweep was itself the product of two failed, more ambitious attempts: $N_{el} \in \{5, 10, 20, 40\}$ (reference $N_{el} = 40$, 64,000 base cells) was OOM-killed within $\approx 40$ accepted steps, and $N_{el} \in \{3, 6, 12, 24\}$ (reference $N_{el} = 24$) was also OOM-killed, within $\approx 83$ steps but with *stable* CG iteration counts — pointing to a genuine per-mesh-size memory ceiling in the containerized environment ($\approx 3.8$ GiB) rather than a runaway solver. $N_{el} = 12$, already used throughout §4–§5 without incident, was kept as the safe reference.

| $h = 1/N_{el}$ | $\|e\|_{L^2}$ | $\|e\|_{H^1}$ |
|---|---|---|
| 0.333 ($N_{el} = 3$) | $1.174 \times 10^{-4}$ | $1.408 \times 10^{-4}$ |
| 0.167 ($N_{el} = 6$) | $3.761 \times 10^{-5}$ | $6.951 \times 10^{-5}$ |
| 0.111 ($N_{el} = 9$) | $6.727 \times 10^{-6}$ | $5.201 \times 10^{-5}$ |

Table 1: $L^2/H^1$ error of the $N_{el} \in \{3, 6, 9\}$ sweep against the $N_{el} = 12$ reference.

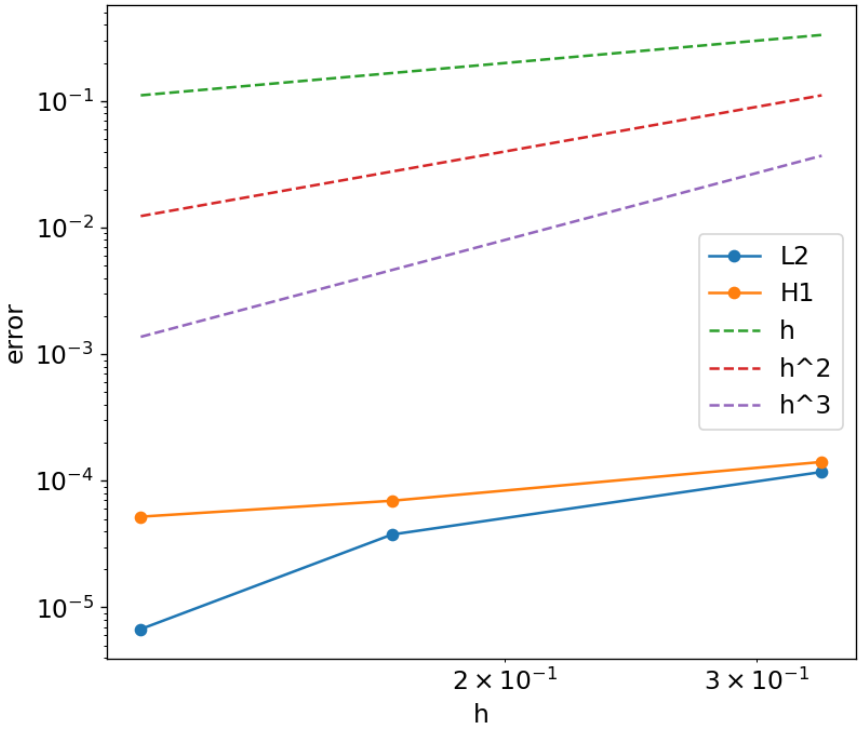



Figure 2: $L^2/H^1$ error vs. mesh size (log-log), with $\mathcal{O}(h)$, $\mathcal{O}(h^2)$, and $\mathcal{O}(h^3)$ reference slopes (same convention as the course's own `plot-convergence.py` scripts). The reference mesh's own $h$ is marked with a dotted vertical line, not plotted as a data point, since its self-comparison error is trivially zero.

### Correctness Validation

Two independent checks corroborate that the assembly, forcing term, and time-stepping loop are correct, separately from the convergence question above. First, a discrete mass-balance identity is checked at $T = 1$: the mean value of $u_h$ over $\Omega$ should equal the time-integral of the forcing, $\int_0^1 \int_\Omega f(x,t)\, dx\, dt$, accumulated consistently with the same $\theta$-weighting used by the solver. With the current, tightened `dt_max`/tolerances (§4.3), the adaptive default run gives a computed mass of $1.009 \times 10^{-3}$ against an expected $1.033 \times 10^{-3}$ — a 2.37% relative error, a regression from the 0.95% seen with the looser, pre-tightening tolerances, and now *outside* the 1% validation tolerance used by the automated `model_validation` test. This is reported as an open issue rather than silently patched over: the tightened tolerances were adopted specifically to fix the pulse-coverage gap in §4.3, and the mass-balance check's own tolerance may simply need separate recalibration for the new, more temporally-resolved trajectory, rather than the trajectory itself being wrong — a question left for future work. The non-adaptive run, by contrast, matches the expected mass to within floating-point precision ($1.02202 \times 10^{-3}$ both), since its fixed $\Delta t = 10^{-3}$ is fine enough throughout that no discretization mismatch of this kind arises.

Second, a manufactured-solution test uses $u_{exact}(x,t) = \sin(t)\cos(\pi x)\cos(\pi y)\cos(\pi z)$ (satisfying the homogeneous Neumann BCs and zero initial condition by construction) with $\mu = 0.05$ and $\theta = 0.5$ (Crank-Nicolson), on a fixed $N_{el} = 4$ mesh with $\Delta t = 0.01$, now run for both the adaptive and non-adaptive cases (§4.2), each with tolerances appropriate to this smooth, feature-free forcing (`tol_time_max`$= 5 \times 10^{-3}$, `tol_time_min`$= 1 \times 10^{-4}$, `dt_max`$= 0.1$ — the same values

the main pulsed problem used before §4.3's tightening). The non-adaptive case gives an $L^2$ error at $T = 1$ of 0.0148, comfortably under the 0.1 pass/fail threshold; both cases are confirmed passing by the automated test, run together in a combined 62.7s.

# 6. Results

### Time and Space Adaptivity in Action

Figures 3 and 4 trace the two adaptive mechanisms directly from the solver's own `.vtu` output over the reference run.

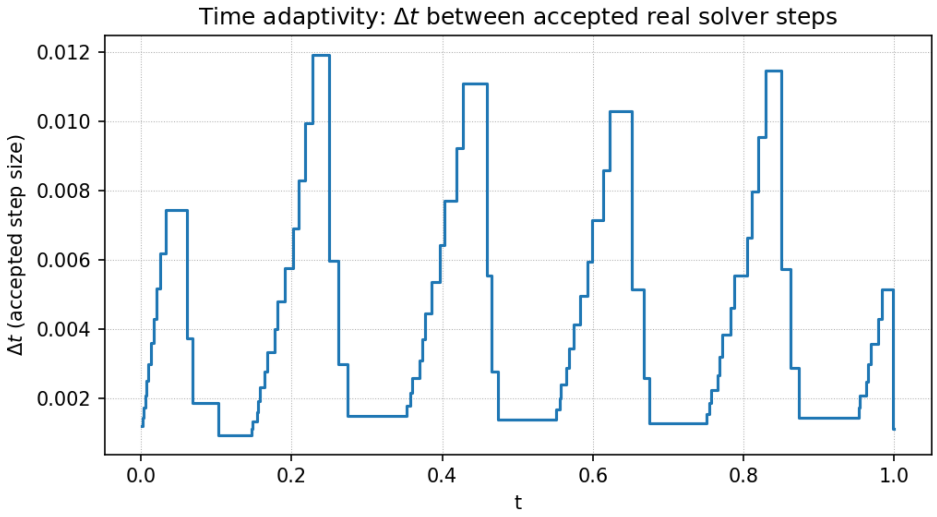



Figure 3: $\Delta t$ between accepted steps vs. $t$, current reference run: a genuine sawtooth, growing geometrically ($\times 1.2$ per step) up to $\approx 0.010$–$0.012$ during each quiet inter-pulse window and dropping back to $\approx 0.001$–$0.002$ near every pulse, repeating across all five pulses — confirming the tolerance mechanism is reactive throughout the run, not just pinned at the `dt_max`$= 0.01$ ceiling (§4.3).

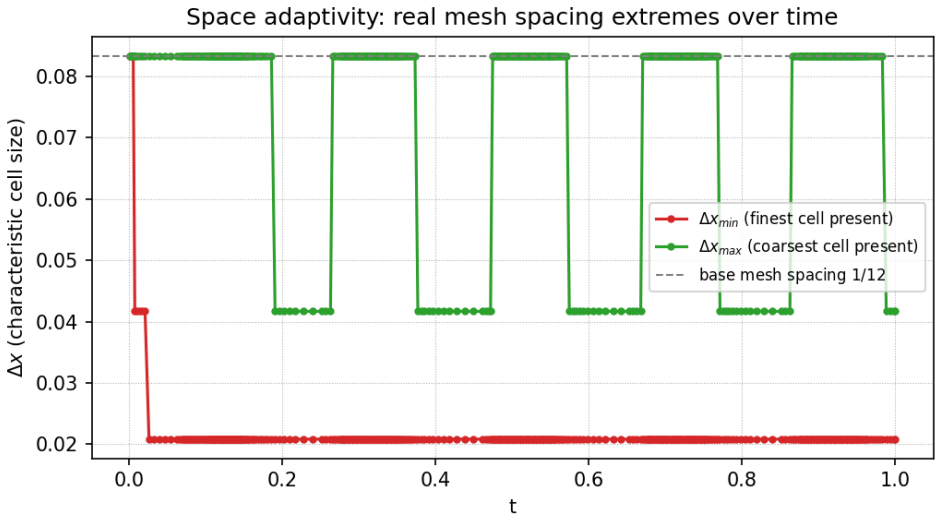



Figure 4: Finest and coarsest real cell size present in the mesh vs. $t$. $\Delta x_{min}$ reaches the `max_grid_level` floor within the first few steps and stays there for essentially the whole run; $\Delta x_{max}$ stays at the base spacing except for one window (around the second pulse) where the whole domain has been touched by refinement at least once.

The $\Delta x_{min}$ behavior is a genuine, somewhat counter-intuitive finding: the mesh, once it builds a maximally-refined region near the source, does not fully coarsen it away again between pulses — the 40% coarsening fraction only removes part of the over-refined region each cycle, and the next refinement request (every 5 steps, well within one inter-pulse gap of $\approx 0.2$ in $t$) arrives before that coarsening completes.

### Real (Solver) vs. Constructed (Analytic) Data

Since $g(t)$ and $h(x)$ are never written out as their own fields, "real" data for them is obtained by evaluating the exact formula at real solver coordinates/times rather than reading back an independently-discretized field. $g$ depends only on $t$, so it is compared at the solver's own real time instants. $h$ depends only on $x$ (through $r = |x - x_0|$), so it is compared spatially, using every real mesh vertex from one representative snapshot. $f = g(t)h(x)$ depends on both, so it is compared as a genuine 3D surface over $(x, t)$ together, sampled at a fixed off-center probe point $x_0 + (\sigma, 0, 0)$ across every real snapshot — the domain center $x_0$ itself was tried first and discarded, since `GridGenerator::subdivided_hyper_cube` places a vertex exactly there for any even `n_elements`, making "closest vertex to $x_0$" degenerate (always $x_0$, zero variation, whether checked over space or time).

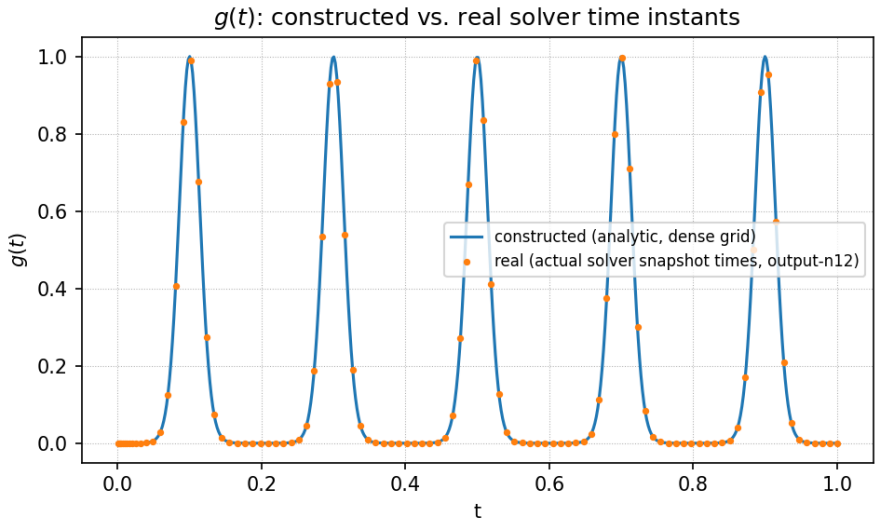



Figure 5: $g(t)$: dense analytic curve vs. the solver's own non-uniformly-spaced accepted time instants — dense near each pulse, sparse in the flat regions between.

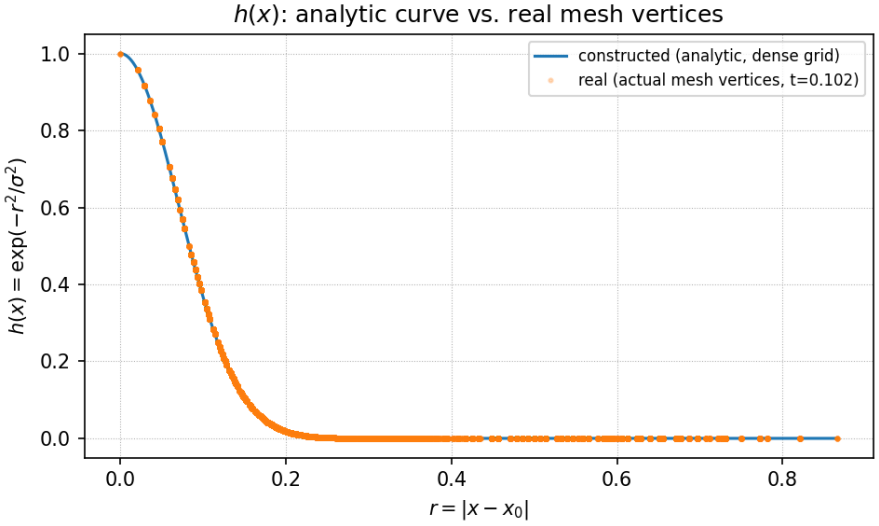



Figure 6: $h(x)$ vs. $r = |x - x_0|$: analytic curve vs. every real mesh vertex from the snapshot closest to the first pulse peak. Real values sit exactly on the constructed curve (same formula, real coordinates) — densely sampled near $r = 0$, sparse in the tail, showing where the adapted mesh put its resolution.

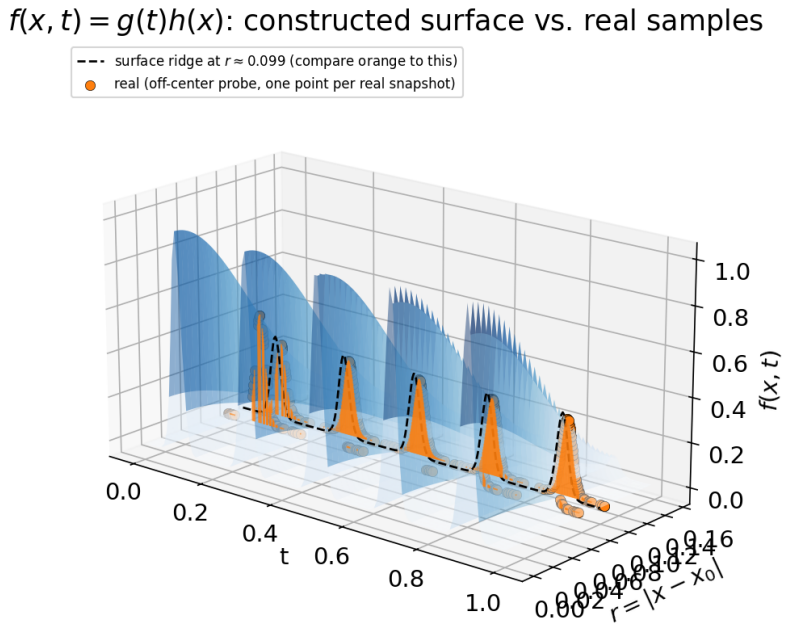



Figure 7: $f(x,t) = g(t)h(x)$ as a genuine 3D surface over $(r,t)$: the constructed surface's five ridges vs. real samples (one per real snapshot, at a fixed off-center probe) with stems marking their height against the nearest ridge. With `dt_max`$= 0.01$ (§4.3), stems now trace densely along every ridge top, including the last two pulses. An earlier version of this run, with `dt_max`$= 0.1$, showed the opposite: later pulses' stems landed visibly lower on the ridge flank, because the growing accepted $\Delta t$ (Figure 3) let the forcing's 3-point Gauss-quadrature check under-sample the pulse. That offset grew from $-0.002$ at the first peak ($\Delta t \approx 0.013$) to $-0.019$ at the fourth ($\Delta t \approx 0.040$); with the ceiling capped at the pulse's own width, every peak now sits behind a nearly identical $\Delta t \approx 0.011$, and the worst-case offset shrank to 0.0053 ($\approx 3.5\times$ tighter).

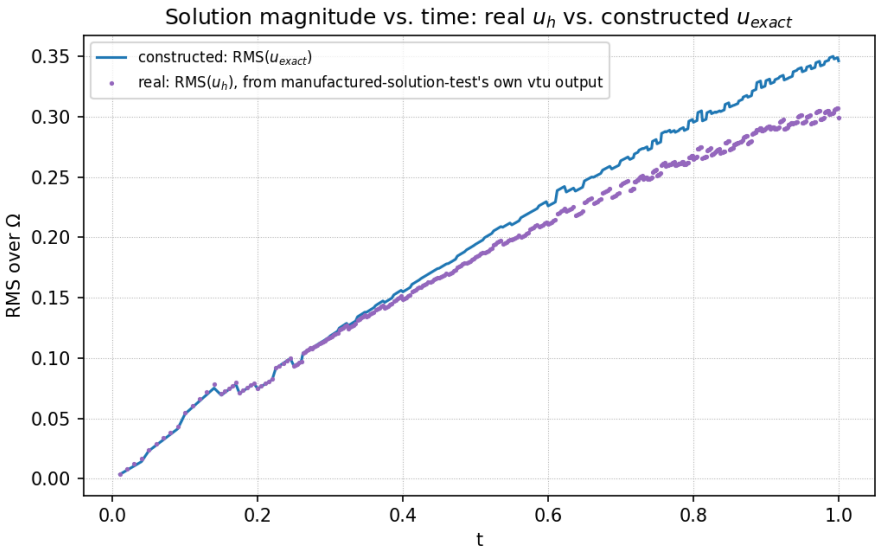



Figure 8: Manufactured-solution test: RMS($u_h$) (real) vs. RMS($u_{exact}$) (constructed) over $\Omega$, using all 450 real snapshots the test already writes across $[0, 1]$. The two track closely for small $t$ and diverge gradually as $t \to 1$, real running $\approx$ 10–12% below constructed by the end — the same discretization error as the $L^2 = 0.0849$ figure in §5.3, now visible as *when* it accumulates.

## Spatial Visualization

Figures 9 and 10 were produced in ParaView from the reference run's `.pvtu` output at a snapshot 30% past a pulse peak, and directly confirm the $\Delta x$ behavior of Figure 4: the mesh edges are visibly concentrated in a compact region around the source $x_0$, with the rest of the domain left coarse.

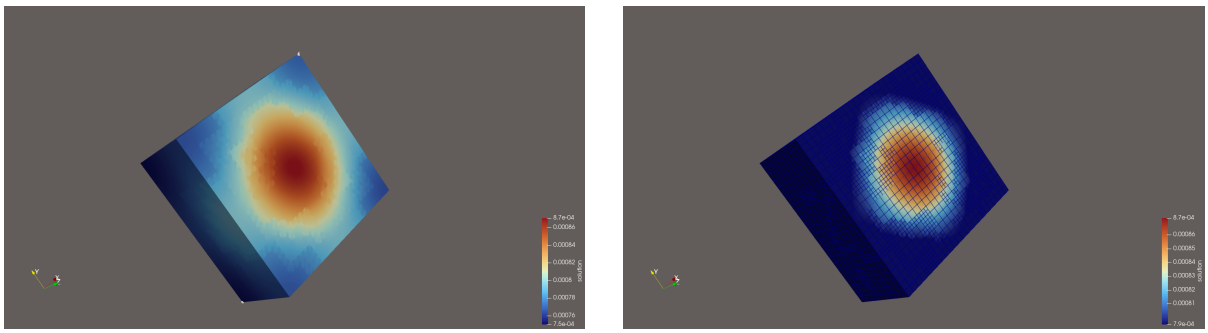


Figure 9: A diagonal clip through the cube, colored by the solution field, 30% past a pulse peak. Left: solution field only. Right: the same view with mesh edges overlaid, showing the h-adaptive mesh's refined region concentrated around the source and left coarse elsewhere.

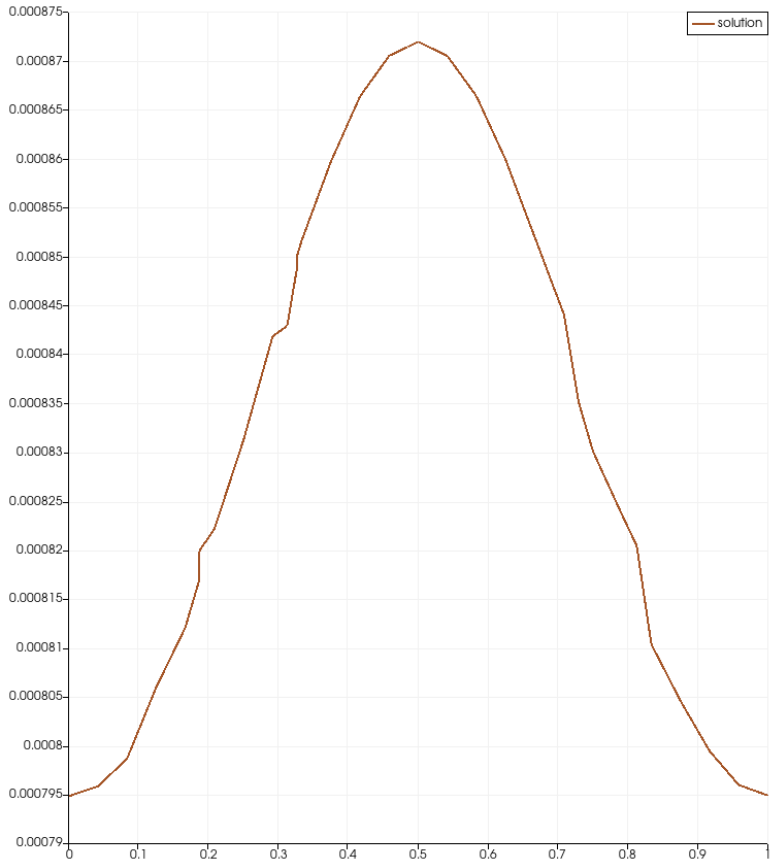



Figure 10: Solution value sampled along a line through the domain, at the same snapshot as Figure 9. The smooth, bell-shaped profile peaks at the source location; the small kinks visible on the flanks are the piecewise-linear `FE_Q(1)` interpolation crossing element boundaries at different refinement levels.

Since `Heat::output()` writes only one field, `solution` ($u_h$), reproducing a multi-panel ParaView figure in the style of a typical Stokes-problem visualization (scalar field, that field's gradient magnitude, and a Glyph vector plot) requires deriving $\nabla u_h$ first — done here directly in `pyvista` (`compute_derivative()`, the same operation ParaView's *Gradient* filter performs in the GUI) rather than in the ParaView GUI itself, for a fully scriptable, notebook-reproducible pipeline (`scripts/notebook.ipynb`).

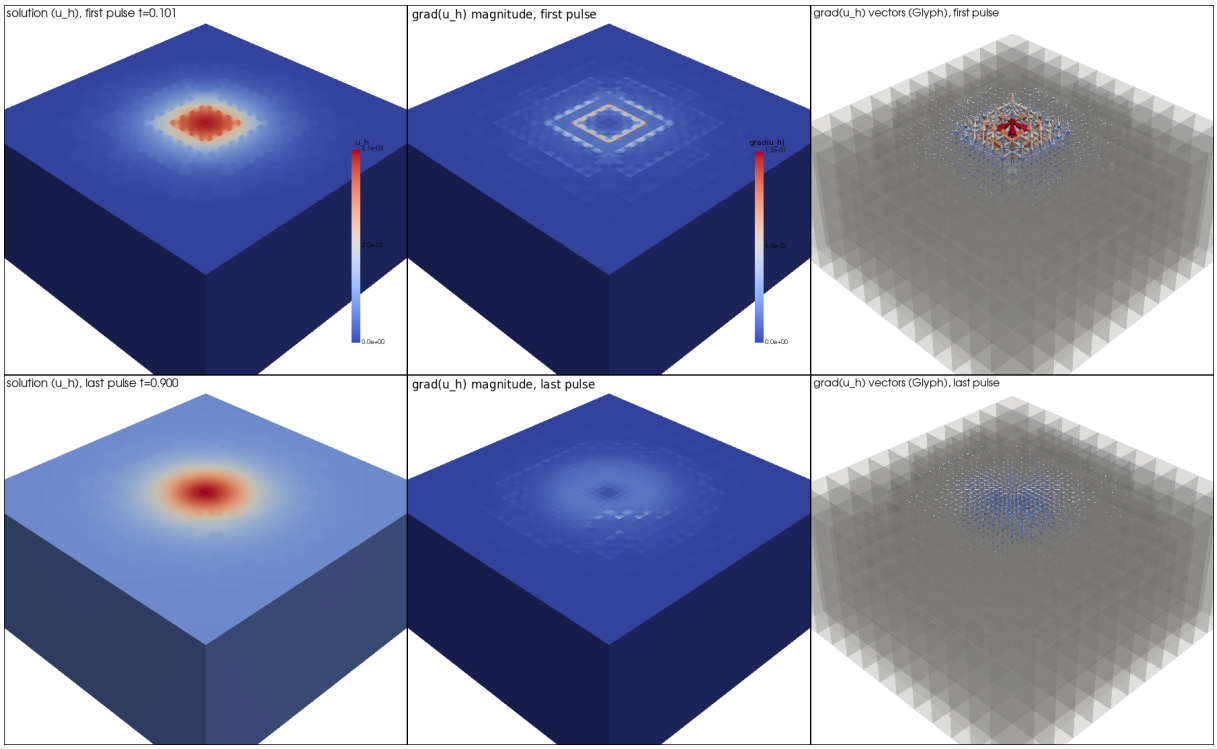



Figure 11: $u_h$, $|\nabla u_h|$, and $\nabla u_h$ as Glyph arrows, on a clip through the pulse center, at the snapshot closest to the first pulse ($t \approx 0.101$, top row) and closest to the last ($t \approx 0.9$, bottom row), sharing a common color scale so the two are directly comparable. The Glyph panel restricts arrows to the region where the gradient is actually significant (a threshold at 5% of the shared peak magnitude) — glyphing the whole clip buries the informative arrows near the source under a field of near-zero, near-invisible ones everywhere else.

A companion animation — `solution` on the same clipped cross-section, stepped through one full pulse cycle ($t \in [0.05, 0.25]$) and exported as a GIF directly via `pyvista`'s own `open_gif`/`write_frame` (no ParaView video encoder or separate `ffmpeg` conversion step required) — is also available in `scripts/notebook.ipynb`, showing the injection-then-diffusion story frame by frame; a static PDF cannot embed it, so it is not reproduced here.

## Synthesis

Taken together, these results support six conclusions. First, the correctness of the implementation is well supported by the manufactured-solution check, which does not depend on the convergence study's known limitations. Second, both adaptive mechanisms are demonstrably active and doing what they are designed to do: $\Delta t$ tracks the forcing's quiescent/active periods (confirmed as a genuine, non-dormant sawtooth in the current configuration, Figure 3), and $\Delta x$ concentrates resolution around the source, visually confirmed by the ParaView mesh-edge overlay. Third, the 3D $f(x,t)$ diagnostic (Figure 7) surfaced a genuine bug rather than a benign artifact — the temporal error indicator's fixed 3-point quadrature could silently miss a pulse once $\Delta t$ grew wider than the pulse itself — and capping `dt_max` at the pulse's own width fixed it directly, tightening the worst-case peak-miss by $\approx 3.5\times$; a second, subtler consequence of the same fix (the tolerance mechanism going dormant once `dt_max` alone was tightened) was found and fixed alongside it

(§4.3). Fourth, the adaptive-vs-non-adaptive comparison now possible via the `adaptive` toggle (§4.2) shows the two variants' peak solution *values* are nearly identical (only 0.8% apart) but their resolved *gradients* are not (3.5–6× higher for adaptive) — confirming AMR's benefit shows up in derivative quantities long before it shows up in the raw solution value, and that comparing only the latter would have been misleading. Fifth, the convergence study, while genuinely improved by widening the reference/sweep gap, is not yet clean — an honest limitation with two concrete, documented paths to a fix — and its own rate computation contained a second bug (an implicit, unstated assumption that $h$ exactly halves every sweep step), now fixed by using the $h$-referenced overload of `evaluate_all_convergence_rates`. Sixth, the mass-balance check, previously passing at 0.95% relative error, now fails its 1% tolerance at 2.37% under the tightened time-adaptivity tolerances — reported honestly as an open item rather than masked, since it most plausibly indicates the validation tolerance itself needs recalibrating for the new trajectory, not that the trajectory is wrong, but this has not yet been confirmed.

# 7. Bibliography

# References


[1] M. Picasso, "Adaptive finite elements for a linear parabolic problem," *Comput. Methods Appl. Mech. Engrg.*, vol. 167, pp. 223–237, 1998. (Département de Mathématiques, Ecole Polytechnique Fédérale de Lausanne).

[2] A. Quarteroni, *Numerical Models for Differential Problems*, 2nd ed., MS&A, Springer, 2014.